

Laborator 9

Lacăte și variabile condiționale

9.1 Scopul lucrării

Laboratorul curent prezintă lacătele (eng. *mutex*) și variabilele condiționale, ca mecanisme de sincronizare în Linux.

9.2 Fundamente teoretice

9.2.1 Conceptul de lacăt

Lacătul este unul dintre cele mai simple mecanisme de sincronizare, având doar două stări: blocat (eng. *locked*) și deblocat (eng. *unlocked*). În general se folosește pentru a sincroniza thread-uri din același proces.

Atunci când un thread dorește să intre într-o regiune critică va încerca să blocheze lacătul (operația *lock*). Dacă reușește, va trece mai departe, iar lacătul va fi blocat. Dacă lacătul este deja blocat, thread-ul curent va fi blocat până ce lacătul se deblochează.

La ieșirea din regiunea critică, thread-ul care a blocat lacătul trebuie să îl și deblocheze. Nici un alt thread nu poate debloca lacătul.

9.2.2 Inițializarea și ștergerea unui lacăt

```
#include <pthread.h>

int pthread_mutex_init(pthread_mutex_t *restrict mutex,
                      const pthread_mutexattr_t *restrict attr);
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

9.2.2.1 pthread_mutex_init

Această funcție inițializează mutex-ul primit ca pointer, folosind atributele din parametrul **attr**. Dacă se dorește utilizarea atributelor implicite, parametrul **attr** poate fi **NULL**. În cazul în care inițializarea s-a efectuat cu succes, funcția returnează valoarea 0, iar lacătul se află în starea *deblocat*.

9.2.2.2 PTHREAD_MUTEX_INITIALIZER

În cazul în care dorim să inițializăm un mutex cu atributele implicite, putem utiliza și macro-ul **PTHREAD_MUTEX_INITIALIZER**, asignând-ul variabilei de tip **pthread_mutex_t**.

9.2.2.3 pthread_mutex_destroy

Această funcție se folosește pentru ștergerea și eliberarea resurselor mutex-ului primit ca parametru.

9.2.2.4 Exemplu

Dorim să scriem un program care pornește 4 thread-uri care execută aceeași funcție. În fiecare thread, avem o regiune comună, care se poate executa în paralel, respectiv o regiune critică, care nu are voie să se execute în paralel. În ambele regiuni se va apela `sleep(1)`.

```

1 | int main()
2 | {
3 |     int i;
4 |     pthread_t tids[NR_THREADS];
5 |     pthread_mutex_t lock;
6 |
7 |     if(pthread_mutex_init(&lock, NULL) != 0) {
8 |         perror("error initializing the mutex");
9 |         return 1;
10 |    }
11 |    for(i=0; i<NR_THREADS; i++) {
12 |        pthread_create(&tids[i], NULL, thread_function, &lock);
13 |    }
14 |    for(i=0; i<NR_THREADS; i++) {
15 |        pthread_join(tids[i], NULL);
16 |    }
17 |    pthread_mutex_destroy(&lock);
18 |
19 |    return 0;
20 | }
```

În funcția `main()` se declară vectorul `tids` care va reține id-urile thread-urilor create (linia 4), respectiv mutex-ul `lock` (linia 5). La linia 7 se face inițializarea mutex-ului, apelând `pthread_mutex_init()`. Mutex-ul se putea inițializa și la declarație: `pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;`.

Thread-urile vor fi create în bucla de la liniile 11–13, apoi se va aștepta terminarea acestora în bucla de la liniile 14–16. Funcția executată de thread primește ca argument doar o referință spre lacăt.

La final, lacătul este distrus (linia 17).

Funcția thread-ului (fără protecția regiunii critice) este prezentată mai jos.

```

1 | void *thread_function(void *arg)
2 | {
3 |     pthread_mutex_t *lock = (pthread_mutex_t*)arg;
4 |
5 |     //common region
6 |     sleep(1);
7 |
8 |     //critical region
9 |     sleep(1);
10 |
11 |    return NULL;
12 | }
```

Vom executa programul folosind comanda `time`:

```
$ time ./l09p1_mutex
```

```
real    0m2,005s
```

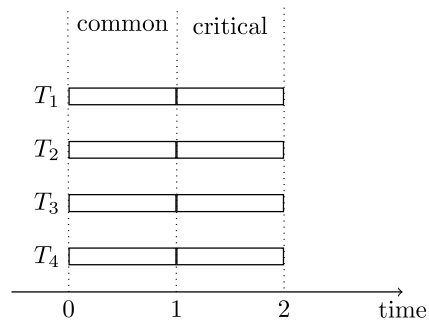


Figura 9.1: Execuția în paralel a thread-urilor fără protejarea regiunii critice

```
user    0m0,001s
sys     0m0,005s
```

Rezultatul comenzii ne indică faptul că cele 4 thread-uri s-au executat în paralel (atât regiunea comună, cât și regiunea critică). Fiecare thread a “dormit” timp de 2 secunde în total, iar cum totul s-a executat în paralel, timpul total de execuție (*real*) a fost de aproximativ 2 secunde, conform Figurii 9.1.

9.2.3 Operații pe lacăte

```
#include <pthread.h>

int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

9.2.3.1 pthread_mutex_lock

Apelul acestei funcții realizează blocarea lacătului. Dacă lacătul este deja blocat, thread-ul curent va fi pus în așteptare până la deblocarea acestuia, de către thread-ul care îl ține blocat.

9.2.3.2 pthread_mutex_unlock

Se realizează deblocarea lacătului. Doar thread-ul care a blocat lacătul are voie să îl deblocheze.

9.2.3.3 pthread_mutex_trylock

Se încearcă blocarea lacătului, la fel ca la `pthread_mutex_lock`. În cazul în care lacătul era liber, comportamentul este identic. În cazul în care lacătul era deja blocat, funcția nu blochează thread-ul curent, ci returnează o valoare diferită de zero.

9.2.3.4 Exemplu: protecția regiunii critice

Vom reveni la exemplul anterior, și vom modifica funcția `thread_function`, astfel încât regiunea critică să fie protejată prin lacăt.

```
1 void *thread_function(void *arg)
2 {
3     pthread_mutex_t *lock = (pthread_mutex_t*)arg;
4
5     //common region
6     sleep(1);
7
8     //critical region
```

```

9 | pthread_mutex_lock(lock);
10 | sleep(1);
11 | pthread_mutex_unlock(lock);
12 |
13 | return NULL;
14 | }

```

Pointer-ul spre lacăt se obține făcând cast argumentului primit la tipul `pthread_mutex_t` (linia 3). Regiunea critică începe prin blocarea lacătului, (linia 9) și se termină prin deblocarea acestuia (linia 11).

Vom executa din nou programul folosind comanda `time`:

```
$ time ./109p1_mutex
```

```

real    0m5,004s
user    0m0,001s
sys     0m0,003s

```

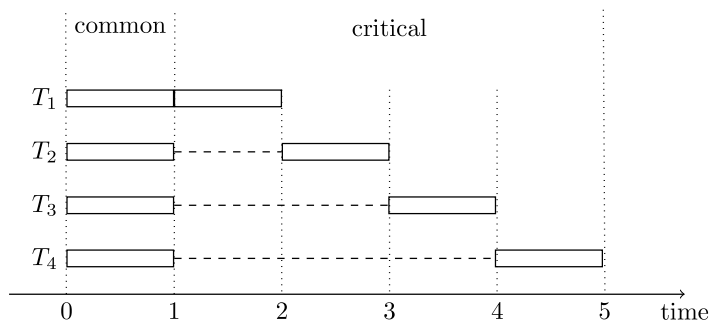


Figura 9.2: Execuția în paralel a thread-urilor cu protejarea regiunii critice

De această dată, timpul total a fost de aproximativ 5 secunde. Regiunea comună a fost executată în paralel, timp de o secundă. Regiunea critică a fost executată de fiecare thread, pe rând, timp de o secundă, deci 4 secunde în total. Figura 9.2 ilustrează execuția thread-urilor. Ordinea de execuție a regiunilor critice în cele 4 thread-uri poate să difere de la o rulare la alta.

9.2.4 Conceptul de variabilă condițională

La unele probleme de sincronizare, un thread trebuie să aștepte îndeplinirea unei condiții, pentru a trece mai departe. Variabilele implicate în condiție sunt protejate de un lacăt. Prin urmare, thread-ul trebuie să blocheze lacătul ca să poată accesa variabilele. Un alt thread, care ar putea modifica variabilele implicate în condiție, are și el nevoie de același lacăt. Dacă thread-ul care așteaptă îndeplinirea condiției ar ține lacătul blocat, s-ar ajunge la o inter-blocare (eng. *deadlock*).

Variabilele condiționale oferă o soluție elegantă pentru problema de mai sus, prin primitiva de așteptare `wait`. Această primitivă poate debloca lacătul pe perioada cât thread-ul curent așteaptă, dând șansa altor thread-uri să modifice variabilele implicate în condiție. Atunci când un alt thread realizează îndeplinirea condiției, poate *semnaliza*, cu primitiva `signal`, variabila condițională, mecanism prin care thread-ul care așteaptă va fi trezit.

9.2.5 Inițializarea și ștergerea unei variabile condiționale

```

#include <pthread.h>

int pthread_cond_init(pthread_cond_t *restrict cond,
                     const pthread_condattr_t *restrict attr);
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

```

```
| int pthread_cond_destroy(pthread_cond_t *cond);
```

9.2.5.1 pthread_cond_init

Această funcție inițializează variabila condițională primită ca pointer, folosind atributele din parametrul `attr`, care poate fi `NULL` dacă dorim atributele implicite.

9.2.5.2 PTHREAD_COND_INITIALIZER

Acest macro se poate folosi pentru inițializarea unei variabile condiționale cu atributele implicite.

9.2.5.3 pthread_cond_destroy

Această funcție eliberează resursele variabilei condiționale primite ca parametru.

9.2.6 Operații pe variabile condiționale

```
#include <pthread.h>

int pthread_cond_wait(pthread_cond_t *restrict cond,
                      pthread_mutex_t *restrict mutex);
int pthread_cond_timedwait(pthread_cond_t *restrict cond,
                           pthread_mutex_t *restrict mutex,
                           const struct timespec *restrict abstime);

int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

9.2.6.1 pthread_cond_wait

Funcția `pthread_cond_wait()` pune thread-ul curent în așteptare, până când se apelează funcția `pthread_cond_signal()` sau `pthread_cond_broadcast()`, cu aceeași variabilă condițională ca parametru al celor două funcții.

Parametrul `mutex` trebuie să fie un pointer spre un lacăt valid, care a fost blocat în prealabil de către thread-ul curent. La intrarea în starea de așteptare a thread-ului curent (în primitiva `wait`), lacătul va fi deblocat, iar la trezirea thread-ului și reluarea execuției lui, dar înainte de revenirea din `wait`, lacătul va fi blocat din nou.

9.2.6.2 pthread_cond_timedwait

Funcția `pthread_cond_timedwait()` are un comportament similar cu `pthread_cond_wait()`, dar mai primește în plus parametrul `abstime`, prin care se specifică un moment de timp în viitor. Dacă până la momentul de timp specificat nu se primește semnalul, thread-ul curent va fi oricum trezit.

9.2.6.3 pthread_cond_signal

Această funcție trezește **primul** thread pus în așteptare de variabila condițională primită ca parametru.

9.2.6.4 pthread_cond_broadcast

Această funcție trezește **toate** thread-urile puse în așteptare de variabila condițională primită ca parametru.

9.2.6.5 Exemplu

O companie are un cont comun, pentru mai mulți angajați, a cărui balanță este reprezentată de variabila globală **balance**. Asupra contului se pot face operațiuni de depunere (*deposit*) și retragere (*withdraw*), fiecare acțiune fiind modelată de un thread separat. Balanța contului nu poate deveni negativă, așa că un thread care vrea să retragă bani trebuie să aștepte ca suma dorită să existe în cont.

Vom avea în total 7 thread-uri:

- thread-urile 0, 2, 3 și 4 vor extrage fiecare câte 7 lei;
- thread-urile 1, 5 și 6 vor depune fiecare câte 11 lei.

Vom folosi un lacăt ca să sincronizăm accesul la variabila **balance**. Thread-urile care nu pot efectua retragerea vor aștepta folosind o variabilă condițională.

```

1  #define NR_THREADS 7
2
3  typedef struct {
4      int id;
5      pthread_mutex_t *lock;
6      pthread_cond_t *cond;
7  } TH_STRUCT;
8
9  int main()
10 {
11     int i;
12     TH_STRUCT params[NR_THREADS];
13     pthread_t tids[NR_THREADS];
14     pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
15     pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
16
17     for(i=0; i<NR_THREADS; i++) {
18         params[i].id = i;
19         params[i].lock = &lock;
20         params[i].cond = &cond;
21         if(i == 0 || i == 2 || i == 3 || i == 4) {
22             pthread_create(&tids[i], NULL, thread_withdraw, &params[i]);
23         } else {
24             pthread_create(&tids[i], NULL, thread_deposit, &params[i]);
25         }
26     }
27
28     for(i=0; i<NR_THREADS; i++) {
29         pthread_join(tids[i], NULL);
30     }
31
32     pthread_mutex_destroy(&lock);
33     pthread_cond_destroy(&cond);
34
35     return 0;
36 }
```

Fiecare thread va primi ca parametru un pointer de tip **TH_STRUCT*** (structură declarată la liniile 3-7), care va conține id-ul curent (un număr între 0 și 6), un pointer la lacăt, respectiv un pointer la variabila condițională. În funcția **main()** se va declara un vector de astfel de structuri (linia 12), câte una pentru fiecare thread, precum și un vector care va reține *thread id*-urile create (linia 13).

Lacătul `lock` și variabila condițională `cond` sunt declarate și inițializate la liniile 14 și, respectiv 15, distrugerea lor făcându-se la finalul programului (liniile 32 și, respectiv 33).

Bucloa de la liniile 17–26 completează parametrii fiecărui thread, apoi creează thread-ul, alegând funcția potrivită, în funcție de id (`thread_withdraw` sau `thread_deposit`). Bucloa de la liniile 28–30 așteaptă terminarea tuturor thread-urilor.

```

1  #define AMOUNT_WITHDRAW 7
2
3  void *thread_withdraw(void *arg)
4  {
5      TH_STRUCT *s = (TH_STRUCT*)arg;
6      usleep(10000 * s->id);
7
8      pthread_mutex_lock(s->lock);
9      while(balance < AMOUNT_WITHDRAW) {
10         printf("[Th%d] Not enough money (%d). Will wait...\n", s->id, balance);
11         pthread_cond_wait(s->cond, s->lock);
12     }
13     balance -= AMOUNT_WITHDRAW;
14     printf("[Th%d] Withdrawn %d. Current balance is %d.\n",
15           s->id, AMOUNT_WITHDRAW, balance);
16     pthread_mutex_unlock(s->lock);
17
18     return NULL;
19 }
```

Funcția `thread_withdraw()` protejează accesul la variabila `balance` folosind lacătul `lock`, care se blochează la linia 8 și se deblochează la linia 16.

Cât timp balanța este mai mică decât suma dorită (linia 9), thread-ul curent trebuie să aștepte ca alte thread-uri să mai depună bani (linia 11). Nu putem aștepta folosind un alt lacăt sau un semafor, deoarece suntem în interiorul regiunii critice, am blocat lacătul `lock`, iar în aceste condiții, nici un alt thread nu ar putea ajunge să modifice balanța. Variabila condițională, în schimb, realizează deblocarea lacătului pe perioada în care thread-ul curent e blocat, permițând altor thread-uri să modifice variabila `balance`.

```

1  #define AMOUNT_DEPOSIT 11
2
3  void *thread_deposit(void *arg)
4  {
5      TH_STRUCT *s = (TH_STRUCT*)arg;
6      usleep(10000 * s->id);
7
8      pthread_mutex_lock(s->lock);
9      balance += AMOUNT_DEPOSIT;
10     printf("[Th%d] Deposited %d. Current balance is %d.\n",
11           s->id, AMOUNT_DEPOSIT, balance);
12     if(balance >= 2 * AMOUNT_WITHDRAW) {
13         pthread_cond_broadcast(s->cond);
14     } else if (balance >= AMOUNT_WITHDRAW) {
15         pthread_cond_signal(s->cond);
16     }
17     pthread_mutex_unlock(s->lock);
18
19     return NULL;
20 }
```

Funcția `thread_deposit()` are un comportament similar cu `thread_withdraw()`, utilizând același lacăt (liniile 8 și, respectiv 17) pentru a sincroniza accesul la balanță. Depunerea de bani

se poate face oricând, deci acest thread nu trebuie să aștepte. Dacă noua balanță ar permite retrageri, eventualele thread-uri care așteaptă să facă acest lucru ar trebui anunțate. În caz că suma este suficientă pentru mai multe retrageri (linia 12), sunt trezite toate thread-urile, prin *broadcast* (linia 13). Dacă suma ajunge doar pentru o singură retragere (linia 14), atunci un singur thread este semnalizat (linia 15).

Faptul că fiecare thread așteaptă la început un timp direct proporțional cu id-ul acestuia (la apelul funcției `usleep()`) crește șansele ca intrarea în regiunea critică să se facă în ordinea id-urilor. Totuși, acest lucru nu este garantat. În a doua problemă propusă, se cere implementarea unui mecanism care să garanteze ordinea intrării thread-urilor în regiunea critică.

Rezultatul rulării programului se vede mai jos:

```
[Th0] Not enough money (0). Will wait...
[Th1] Deposited 11. Current balance is 11.
[Th0] Withdrawn 7. Current balance is 4.
[Th2] Not enough money (4). Will wait...
[Th3] Not enough money (4). Will wait...
[Th4] Not enough money (4). Will wait...
[Th5] Deposited 11. Current balance is 15.
[Th2] Withdrawn 7. Current balance is 8.
[Th4] Withdrawn 7. Current balance is 1.
[Th3] Not enough money (1). Will wait...
[Th6] Deposited 11. Current balance is 12.
[Th3] Withdrawn 7. Current balance is 5.
```

- Thread-ul 0 dorește să retragă 7 lei, dar balanța este 0, așa că este pus în așteptare.
- Thread-ul 1 depune 11 lei, apoi semnalizează thread-ului 1, care se deblochează și face retragerea.
- Thread-urile 2, 3 și 4, încearcă pe rând să retragă 7 lei, dar balanța este 4, așa că cele 3 thread-uri vor fi blocate.
- Thread-ul 5 depune 11 lei, ducând balanța la 15. Deoarece din această sumă se pot face mai multe retrageri, se va face *broadcast*, thread-urile 2, 3 și 4 fiind trezite. Primele două dintre acestea (în acest caz 2 și 4) reușesc să retragă bani, ultimul (3) fiind din nou pus în așteptare.
- Thread-ul 6 depune și el 11 lei, apoi trezește thread-ul 3 care face retragerea.

9.3 Mersul lucrării

9.3.1 Întrebări recapitulative

1. Care este diferența dintre un lacăt și un semafor?
2. Ce opțiuni avem pentru a inițializa un lacăt? Dar pentru o variabilă condițională?
3. De ce funcția `pthread_cond_wait` primește și un lacăt (mutex) ca parametru? Ce va face cu acel lacăt?
4. Care este diferența dintre *signal* și *broadcast* pentru o variabilă condițională?
5. De ce pattern-ul comun de utilizare a unei variabile condiționale presupune verificarea condiției cu `while`, nu cu `if`?

9.3.2 Probleme rezolvate

1. (*l09p1_mutex.c*) Să se scrie un program care pornește mai multe thread-uri ce execută aceeași funcție. Fiecare thread va avea o regiune comună, care se poate executa în paralel, respectiv o regiune critică, care nu se poate executa. Programul este descris în secțiunile 9.2.2.4 și 9.2.3.4.

2. (*l09p1_cond_var.c*) Să se scrie un program care modelează operațiile de depunere și retragere asupra unui cont, prin thread-uri. La operația de retragere, în cazul în care fondurile sunt insuficiente, trebuie să se aștepte completarea acestora printr-o operație de depunere. Programul este descris în secțiunea 9.2.6.5.

9.3.3 Probleme propuse

1. (25p) Se dă un vector **v** de numere întregi, împreună cu dimensiunea acestuia **n**, ca variabile globale. Se dorește construcția unui alt vector **nrDiv** (tot variabilă globală), care să conțină numărul de divizori pentru fiecare element din vector. Pentru asta, vom porni un număr de thread-uri (primit ca argument în linia de comandă), fiecare thread procesând într-o buclă elemente din vectorul **v**, până când se ajunge la finalul acestuia. Thread-urile trebuie sincronizate, astfel încât un element să nu fie procesat de mai multe ori.
2. (25p) Modificați exemplul din secțiunea 9.2.6.5, astfel încât ordinea în care thread-urile intră în regiunea critică să fie garantat cea a id-urilor. Pentru aceasta, se va folosi o variabilă globală numită **turn**, care va indica al cui thread e rândul. Fiecare thread va aștepta, utilizând o variabilă condițională, ca variabila **turn** să devină egală cu id-ul lui.
3. (25p) Implementați un program în care mai multe thread-uri citesc câte un octet dintr-un fișier. Fiecare thread va deschide fișierul apelând funcția **open_once()**. La primul apel al funcției se va realiza deschiderea fișierului, urmând ca la următoarele apeluri să se returneze descriptorul de fișier obținut anterior. Deschiderea fișierului trebuie sincronizată, astfel încât, chiar dacă două thread-uri apelează **open_once()** deodată, să se realizeze o singură deschidere a fișierului.
Bonus: (5p) Funcția **open_once()** va primi ca parametru numele fișierului și se va asigura că fiecare fișier primit se deschide o singură dată.
4. (25p) Implementați un protocol pentru traversarea unui pod cu o singură bandă pentru mașini. În orice moment, mașinile pot traversa podul într-o singură direcție. Cel mult *N* mașini se pot afla simultan pe pod. Fiecare mașină va fi implementată printr-un thread care execută codul de mai jos:

```

1 | void *car_thread(void *direction)
2 | {
3 |     int dir = (int)direction;
4 |     enter_bridge(dir);
5 |     cross_bridge(dir);
6 |     exit_bridge(dir);
7 |     return NULL;
8 | }
```

